

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №2
по курсу «Логика и ОА в ИЗ»
на тему «Сложные структуры данных и алгоритмы их обработки»»

Выполнили:
студенты группы 25ВВВ1
Новиков И.А.
Евстратов В.С.

Принял:
к.т.н., доцент Юрова О.В.

Пенза 2026

Цель работы

Изучение способов организации динамических структур данных (связные списки, стек, очередь, бинарные деревья) на языке Си, освоение методов выделения и освобождения динамической памяти, а также алгоритмов обхода, поиска, добавления и удаления элементов в линейных и нелинейных структурах данных.

Лабораторное задание

Разработать программу, реализующую:

Задание 1: Создание и обработку однонаправленного связного списка целых чисел (добавление элементов, удаление заданного элемента, поиск значения).

Задание 2: Реализацию структуры данных «Стек» на основе динамического связного списка с функциями push, pop и просмотром верхнего элемента.

Задание 3: Реализацию структуры данных «Очередь» на основе динамического связного списка с операциями добавления в конец и извлечения из начала.

Задание 4: Построение бинарного дерева поиска (BST) и реализацию алгоритмов его обхода (прямой, симметричный, обратный).

Описание метода решение задач

Задание 1 (task1):

Описывается структура узла однонаправленного списка `struct Node { int data; struct Node* next; }`. Элементы добавляются динамически с помощью `malloc()`. Поиск элемента выполняется последовательным перебором узлов от головы списка `head` до конца (`NULL`). При удалении узла корректируются указатели соседних элементов, после чего освобождается память удаляемого узла функцией `free()`.

Задание 2 (task2):

Стек организуется по принципу LIFO (Last In, First Out). Элемент добавляется в вершину стека (`push`), при этом новый узел ссылается на предыдущую вершину. Операция `pop` извлекает вершину, сдвигая указатель вершины на следующий элемент и освобождая память.

Задание 3 (task3):

Очередь организуется по принципу FIFO (First In, First Out) с помощью двух указателей: `head` (начало) и `tail` (конец). Функция добавления в очередь

выделяет память под новый узел и привязывает его к tail. Функция извлечения забирает элемент из head и смещает указатель головы очереди.

Задание 4 (task4):

Описывается структура узла бинарного дерева `struct TreeNode { int key; struct TreeNode left; struct TreeNode right; }`. Вставка элементов осуществляется по правилу: меньшие значения направляются в левое поддерево, большие или равные — в правое. Обход дерева реализован рекурсивными функциями:

- *In-order* (левое поддерево -> корень -> правое поддерево);
- *Pre-order* (корень -> левое поддерево -> правое поддерево);
- *Post-order* (левое поддерево -> правое поддерево -> корень).

суммирования элементов `a[i][j]` по индексам строк `i` при фиксированном индексе столбца `j`. По окончании работы высвобождается память всех строк и массива указателей.

Пояснительный текст к программе

Динамические структуры данных: Использование указателей позволяет формировать гибкие структуры данных произвольного размера, меняющиеся непосредственно во время выполнения программы.

Управление памятью: Все создаваемые узлы списка, стека, очереди и дерева выделяются динамически через `malloc()`. По завершении работы программы или удалении элементов выполняется корректное освобождение памяти с помощью `free()`, что исключает утечки памяти.

Рекурсивная обработка деревьев: Алгоритмы обхода и поиска в бинарном дереве реализованы через рекурсивные вызовы функций, что обеспечивает компактность и наглядность кода.

Результаты работы программы

```
PS C:\Users\bxd\Documents\GitHub\LiOAvIZ\lab2> gcc -Wall -Wextra -ansi -pedantic -o lab2.exe lab2.c; .\lab2.exe
lab2
1 - matrix mul
2 - sort alg
> 1
matrix mul, O(n^3)

      n      time (s)
-----
      100      0.0010
      200      0.0100
      400      0.0830
     1000      1.3660
     2000     15.8300
     4000    160.6690
    10000   3529.1140

-- graph --
 100 | 0.0010
 200 | 0.0100
 400 | 0.0830
1000 | 1.3660
2000 | 15.8300
4000 | ## 160.6690
10000 | ##### 3529.1140
```

```
PS C:\Users\bxd\Documents\GitHub\LiOAvIZ\lab2> gcc -Wall -Wextra -ansi -pedantic -o lab2.exe lab2.c; .\lab2.exe
lab2
1 - matrix mul
2 - sort alg
> 2
sort alg timing

--- n = 1000 ---
Random      Shell: 0.000000 QuickSort: 0.000000 qsort: 0.000000
Ascending   Shell: 0.000000 QuickSort: 0.000000 qsort: 0.000000
Descending   Shell: 0.000000 QuickSort: 0.000000 qsort: 0.000000
Mountain    Shell: 0.000000 QuickSort: 0.000000 qsort: 0.000000

--- n = 5000 ---
Random      Shell: 0.001000 QuickSort: 0.000000 qsort: 0.001000
Ascending   Shell: 0.000000 QuickSort: 0.000000 qsort: 0.000000
Descending   Shell: 0.002000 QuickSort: 0.000000 qsort: 0.000000
Mountain    Shell: 0.001000 QuickSort: 0.003000 qsort: 0.001000

--- n = 10000 ---
Random      Shell: 0.006000 QuickSort: 0.001000 qsort: 0.000000
Ascending   Shell: 0.000000 QuickSort: 0.000000 qsort: 0.000000
Descending   Shell: 0.007000 QuickSort: 0.000000 qsort: 0.000000
Mountain    Shell: 0.005000 QuickSort: 0.011000 qsort: 0.001000

--- n = 50000 ---
Random      Shell: 0.142000 QuickSort: 0.003000 qsort: 0.004000
Ascending   Shell: 0.000000 QuickSort: 0.001000 qsort: 0.001000
Descending   Shell: 0.274000 QuickSort: 0.000000 qsort: 0.001000
Mountain    Shell: 0.132000 QuickSort: 0.297000 qsort: 0.003000

--- n = 100000 ---
Random      Shell: 0.580000 QuickSort: 0.008000 qsort: 0.007000
Ascending   Shell: 0.001000 QuickSort: 0.001000 qsort: 0.002000
Descending   Shell: 0.993000 QuickSort: 0.001000 qsort: 0.002000
```

Выводы

В ходе выполнения лабораторной работы №2 были изучены:

1. Принципы организации сложных динамических структур данных (динамические списки, стеки, очереди, бинарные деревья) на языке Си.
2. Методы управления динамической памятью при работе с указателями в связанных структурах.
3. Практические алгоритмы вставки, поиска, удаления и рекурсивного обхода элементов связанных списков и бинарных деревьев поиска.